

Vortex versioning

Model, proof obligations, and compatibility theorems

This document proves the compatibility guarantees described in the Versioning documentation. Under the stated assumptions, every successful write constrained to a set of editions can be read with the same meaning by any reader that supports those editions, including a reader older than the writer.

These are mathematical proofs about a model. They are not a machine-checked proof of the Rust implementation. The assumptions require correct readers and writers, fixed serialized formats, and retained support for those formats in later versions of the code. The implementation section identifies the remaining work. In particular, configuring compression schemes for each target edition is still planned.

1 Compatibility and writer invariants

I1 through I7 establish compatibility for successful writes and require later versions of the reader code to preserve it. I8 through I10 require the writer to retain supported targets, select the oldest suitable format, and avoid recompression solely to meet an edition's constraints.

- I1. **A frozen wire contract is immutable.** A typed wire ID always identifies the same valid forms and their meanings. A reader-visible extension requires a new ID.
- I2. **Readers implement each supported contract.** Exact-ID dispatch and local decoding accept every valid form of that ID, preserve its meaning, and compose correctly with decoded children.
- I3. **Serialization preserves meaning.** Every emitted node satisfies its claimed wire contract, and its meaning equals the meaning of the input representation.
- I4. **Edition enforcement covers the whole output.** A successful constrained write contains only permitted typed IDs, including children, layouts, extension dtypes, and aggregates.
- I5. **Frozen edition records are immutable and membership is cumulative within a family.** Membership, origin, and recorded minimum version remain fixed. A later edition adds permissions without changing the earlier edition. Selecting families takes their union.
- I6. **Reader evolution preserves historical support.** A later conforming version of a project's code retains support for its frozen formats, even after writers stop using them.
- I7. **A frozen edition's origin and minimum version are sound.** That version of the origin's code supplies readers for every member of the edition. Required plugins must be present and registered.
- I8. **Supported writer targets remain writable.** A writer retains the behavior needed to construct permitted representations for the input domain it promises to write to an edition.
- I9. **Serialization chooses the oldest available lossless form.** Selection depends on the representation and the plugin's wire history. Edition validation follows selection. The serializer does not choose a different form by consulting the edition allowlist.
- I10. **Future scheme configuration is consistent and closed under children.** For a target edition, estimation, sampling, and full compression use one compatible behavior configuration. Every produced child is subject to the same capability constraint.

I1 fixes the meaning of a wire ID. I2 constrains readers, I3 constrains serializers, and I4 constrains which IDs a write can emit. I5 fixes the meaning of an edition name. I6 preserves support across versions of the reader code, and I7 connects that support to a published version number. I8 concerns write availability, I9 concerns which valid form is chosen, and I10 prevents compression from producing a form that must be recompressed solely to meet the edition. None follows from the others.

2 Definitions and scope

2.1 Symbols

Symbol	Meaning
K	Component kinds: array, layout, extension dtype, and aggregate.
$u = (k, i)$	A typed wire ID: kind $k \in K$ and identifier string i .
C_u, φ_u	The valid local forms of u , and their semantic interpretation.
$t, U(t)$	A finite serialized object tree, and all typed IDs occurring in it.
$V(t)$	The semantic meaning of a valid serialized tree.
M_L, μ_L	Configuration L 's in-memory representations and their semantic meanings.
R_L, H_L	IDs implemented by reader L , and IDs its writer can emit.
D_L, S_L	Recursive reading and serialization for configuration L .
$e, A(e)$	An edition, and its cumulative set of permitted typed IDs.
$E, A(E)$	A selection of editions, and the union of their permissions.
$o(e), m(e)$	The origin project and its minimum code version recorded for a frozen edition.
$W_{L,E}$	A write with configuration L that enforces selection E .
$\perp$	Failure, rejection, or absence of a result, as specified by the operation.

An ID is typed because identical strings in different kinds identify different contracts. In particular, (array, vortex.flat) and (layout, vortex.flat) are different IDs. No ordering is inferred from ID strings. A plugin explicitly orders its own historical forms.

2.2 Serialized objects and semantic meaning

A node has the form

$$t = (u, p, t_1, \dots, t_n). \quad (1)$$

Here p includes all non-child data needed to interpret the node: metadata, buffers, dtype, length, and any relevant context supplied by its container. Dependencies such as extension dtypes and serialized aggregate functions are included in the tree even when their concrete bytes occur elsewhere. A file has a fixed-format root whose children include all versioned components. That root has a separately assumed stable, correctly implemented contract.

A concrete file can share children. Unfolding its finite acyclic dependency graph into a finite tree does not change the argument. Cyclic graphs, malformed references, corrupted bytes, resource exhaustion, and incompatible file envelopes are outside this compatibility theorem. Finite input and terminating local routines are explicit premises. Permission to use an ID does not establish termination or a resource bound.

A local contract C_u is a predicate on the payload, child interfaces, and child meanings. Interfaces include the structural facts needed by the parent, such as child count, dtype, and length. Its interpretation φ_u defines the meaning of each valid local form. Validity and meaning are defined recursively:

$$V(t) = \varphi_u(p, V(t_1), \dots, V(t_n)). \quad (2)$$

This expression is defined only if each child is valid and the parent satisfies C_u . The semantic domains are sorted by component kind and logical type. For example, an array means its typed values and nulls, an extension dtype means its logical interpretation, and an aggregate means its function and options. A file's meaning includes the declared component behavior as well as its logical data. This prevents silently dropping a configured aggregate from counting as serialization of the same file specification.

The equations omit child interfaces for readability. Local implementations must preserve them as well as semantic values. An aggregate contract's meaning includes the conditions that make its use for pruning sound. The proof does not establish an aggregate algorithm's soundness.

The ID closure is

$$U(t) = \{u\} \cup \bigcup_{j=1}^n U(t_j). \quad (3)$$

The fixed-format root contributes no edition-governed ID. An unrecognized dependency is not removed from $U(t)$ merely because a particular query does not use it.

2.3 Reader and writer configurations

A **configuration** L identifies the versions of the Vortex Rust crates and optional plugin code, the enabled modules, and the registered implementations. A crate version alone does not specify which components an application can read or write.

Its memory domain M_L can differ completely from another configuration's domain. The meaning of $a \in M_L$ is $\mu_L(a)$. The reader set R_L is a set of typed IDs with implementations, not a set of registered edition declarations. The writer set H_L is defined independently: a historical ID can be retained only for reading.

No in-memory version field is assumed. Fields, children, and metadata can distinguish the shapes handled by one current implementation. A deserializer can construct that implementation directly or construct another equivalent current representation. Its output need not have the same in-memory encoding ID as the wire ID that dispatched it.

2.4 Editions, families, and origin versions

An edition selection E contains at most one edition per family. Its permission set is

$$A(E) = \bigcup_{e \in E} A(e). \quad (4)$$

The union of an empty selection is empty. An empty per-kind allowlist permits no component of that kind. Within one family, $e \leq e'$ implies $A(e) \subseteq A(e')$. There is no chronology comparison between editions from different families.

For a frozen edition, $o(e)$ names the **origin**: the project that supplies its component implementations. The minimum version $m(e)$ refers to that project's code. For origin vortex, this is the shared version of the Vortex Rust crates. Independent plugins can have their own origins and version numbers. Version ordering is used only within one origin.

Draft editions have a permission set but no published minimum version or perpetual read-support guarantee. A stable edition can freeze when its origin publishes the code that supports it. Recording the minimum version afterward documents that freeze, rather than creating a new one.

A reader supports a selection when

$$A(E) \subseteq R_L. \quad (5)$$

This is only a coverage statement. Correct decoding follows from the local implementation premise below and an induction, not from this definition alone. Registering an edition declaration changes which permissions can be selected. It does not enlarge R_L , H_L , or M_L .

3 Implementation and publication premises

3.1 Local reader premise (I2)

For every $u \in R_L$, reader L has an exact-ID decoder $d_{L,u}$. Given a valid local payload and correctly decoded child representations $a_1, \dots, a_n$, this decoder terminates and returns $a \in M_L$ with the required interface and

$$\mu_L(a) = \varphi_u(p, \mu_L(a_1), \dots, \mu_L(a_n)). \quad (6)$$

The decoder validates the contract of the exact supplied ID. Recognizing a successor ID does not expand what an older ID permits. In strict reading, dispatch fails when $u \notin R_L$.

This is a local obligation for each implementation. It is stronger than recognizing a string in a registry and weaker than assuming the whole-file compatibility result.

3.2 Local writer premise (I3)

A successful local serializer for $a \in M_L$ returns an ID $u \in H_L$, a payload p , and child representations $a_1, \dots, a_n$. Their interfaces satisfy C_u , and

$$\mu_L(a) = \varphi_u(p, \mu_L(a_1), \dots, \mu_L(a_n)). \quad (7)$$

Recursive serialization follows a finite, well-founded dependency structure. Each child satisfies the same premise. Newly constructed children are covered too. Thus local structural adaptation cannot bypass either the semantic obligation or recursive validation.

Compression, layout construction, or another preparation step used before serialization has its own value-preservation obligation. A theorem about serialization of a guarantees the meaning of a . It guarantees the original source values only when preparation preserved them.

3.3 Constrained-write premise (I4)

The constrained writer recursively checks every emitted typed ID against $A(E)$. It reports success only if serialization and every such check succeed. A failure can occur after some bytes have been written: this premise is about a successfully completed file, not transactional or atomic I/O. Edition checks must use the serializer's returned wire ID rather than the in-memory encoding ID.

3.4 Publication premises (I1, I5, I6, I7)

Once frozen, both C_u and φ_u remain fixed for each published ID. Frozen edition membership, origin, and recorded minimum version also remain fixed. Later editions are cumulative in their own families.

For each frozen e , version $m(e)$ of the origin project's code provides conforming implementations for every member of $A(e)$. Later conforming versions retain that support and its meanings. The project must preserve this support when publishing code. Increasing a version number alone does not establish compatibility. When origins are combined, their implementations must be installed in a compatible host configuration and agree on any shared contracts. Taking the maximum of version numbers from unrelated origins is undefined.

4 Compatibility proofs

4.1 Lemma 1: recursive reading preserves meaning

Let t be valid, finite, and acyclic, and let $U(t) \subseteq R_L$. Under I1 and the local reader premise, $D_L(t)$ succeeds and

$$\mu_L(D_L(t)) = V(t). \quad (8)$$

Proof. Induct on the height of t . A leaf has no children. Its ID belongs to R_L , so exact-ID dispatch selects its decoder. The local reader premise returns the leaf's prescribed meaning. For a non-leaf, each $U(t_j)$ is contained in $U(t)$ and hence in R_L . Each child has smaller height, so the induction hypothesis supplies its correctly decoded representation and interface. The parent's ID also belongs to R_L . Applying Equation 6 to those children yields Equation 2. Finite height and terminating local routines complete the induction. $\square$

This proves closure across component kinds as well as nested arrays. Checking only a root array ID does not supply the induction hypothesis for its child encodings, extension types, or dependencies.

4.2 Lemma 2: recursive serialization preserves meaning

If recursive serialization of a succeeds with tree t , the local writer premise implies that t is valid and $V(t) = \mu_L(a)$.

Proof. Induct on the finite serialization dependency structure. For a leaf, Equation 7 gives both validity and equality. For a parent, the induction hypotheses replace every child meaning $\mu_L(a_j)$ with $V(t_j)$. The local contract establishes parent validity, and Equation 7 becomes Equation 2 with result $\mu_L(a)$. $\square$

4.3 Theorem 1: successful edition-constrained writes are compatible

Let W be any writer configuration, L any conforming reader, and E a selected edition set. If

$$W_{W,E}(a) = t \neq \perp \quad \text{and} \quad A(E) \subseteq R_L, \quad (9)$$

then

$$D_L(t) \neq \perp \quad \text{and} \quad \mu_L(D_L(t)) = \mu_W(a). \quad (10)$$

Proof. Recursive enforcement gives $U(t) \subseteq A(E)$. Coverage then gives $U(t) \subseteq R_L$. Lemma 2 establishes validity and $V(t) = \mu_W(a)$. Lemma 1 gives $\mu_L(D_L(t)) = V(t)$. Transitivity proves the claim. $\square$

The writer can use newer Vortex crates than the reader. No premise compares their crate versions, memory layouts, or compression implementations. The reader needs the emitted contracts, not knowledge of the writer. Two writers can produce different bytes for equal values while both satisfy this theorem. Byte-for-byte reproducibility does not follow.

The theorem is conditional on successful writing. It does not assert that every array has a permitted form, or that every configured compressor can construct one.

4.4 Theorem 2: later readers preserve readability

Suppose t uses frozen IDs, and a conforming reader L reads it under Lemma 1. If L' is a later conforming configuration retaining those implementations under I6, then L' reads t with the same meaning, even if M_L and $M_{L'}$ differ.

Proof. Retention gives $U(t) \subseteq R_{L'}$. I1 preserves the validity and meaning of t . Apply Lemma 1 to L' and to L . Both results have meaning $V(t)$. $\square$

A reader need not convert a historical in-memory object into a current one. It can decode the historical bytes directly into its current representation. Historical contract support is required. A separate legacy memory type, version flag, upgrade pass, or recompression pass is not.

4.5 Theorem 3: frozen edition bounds are sufficient, but conservative

For each origin o named by a frozen selection, define

$$b_o(E) = \max\{m(e) : e \in E, o(e) = o\}. \quad (11)$$

A compatible reader configuration containing each required origin at version $b_o(E)$ or later, with its implementations registered, reads every successful write constrained to E .

Proof. I7 supplies $A(e)$ at $m(e)$ and I6 retains it at the selected later version. Taking the union gives $A(E) \subseteq R_L$. Apply Theorem 1. $\square$

This is a sufficient bound computed from the recorded edition minima for that selection, not a proof of a necessary or globally minimal version of the reader code. A file's actual requirement is $U(t)$, and I4 gives $U(t) \subseteq A(E)$. For example, selecting an edition that permits v1 and v2 but emitting only v1 produces a file an appropriately configured v1-only reader can read. An edition's minimum covers all its members, including ones the file never uses.

Under I5, replacing an edition by a later one in the same family only enlarges the permission set. Consequently a previously valid serialized tree remains permitted. This does not prove that a compressor makes identical choices, nor that the enlarged target retains the same minimum reader.

4.6 Theorem 4: unsupported and forbidden IDs fail at their boundaries

If a candidate output t contains $u \notin A(E)$, it cannot be a successful constrained write. If strict full reading encounters $u \notin R_L$, it fails at dispatch. A writer cannot emit a valid local form through an implementation it lacks, $u \notin H_L$.

Proof. The first conclusion is the contrapositive of recursive enforcement. The second is the exact-ID dispatch rule. The third follows from the definition of H_L and the local writer premise. These are different failures: adding permission cannot supply an implementation, and adding an implementation cannot supply permission. $\square$

Unknown-component passthrough does not satisfy Lemma 1's full semantic decoding result. It can preserve inert bytes for inspection or copying. Ignoring an unknown aggregate can disable pruning and still allow correct logical data reads. That weaker operation is outside the full-component interpretation proved here. Disabling editions removes I4, so Theorem 1's edition conclusion no longer follows, although a specific file can still be readable.

5 Representation changes and writer policy

5.1 Structural adaptation preserves compatibility

Let a plugin adapt a into another representation b at the serialization boundary, with $\mu_W(b) = \mu_W(a)$. If $S_W(b) = t$, Lemma 2 gives

$$V(t) = \mu_W(b) = \mu_W(a). \quad (12)$$

Thus a lossless structural downgrade preserves the compatibility theorem whenever the resulting tree is permitted. Alternatively, the plugin can return adapted payload and child parts directly. The local writer premise yields the same equality without constructing a legacy memory type.

This result is about meaning. The equality does not prove that the transformation is cheap, that it avoids decoding, or that it qualifies as structural rather than recompression. Those are separate operational claims. A plugin must not relabel a newer payload with an older ID unless the payload actually satisfies that older contract.

5.2 Oldest available lossless form

For a current representation a , let $Q_W(a)$ be the finite set of local wire forms the plugin can produce losslessly by its supported representation-preserving serialization operations, without recompression. This set includes only forms the writer implements. Within the relevant wire history, each form has an explicit chronological rank. I9 requires choosing a form with the minimum rank in $Q_W(a)$, or reporting no serialization if the set is empty.

Proposition. If I9 selects form q , no older form in $Q_W(a)$ was skipped. If all IDs in the completed serialization, including those of its children, belong to $A(E)$, the constrained write passes the edition checks.

Proof. The first claim is the defining property of a minimum in a finite ordered set. The second is recursive enforcement applied to the actual output. $\square$

The useful implementation obligation is constructing a sound candidate set and selecting its minimum. Merely listing supported IDs oldest-first does not establish I9. Historical IDs retained only for reading need not belong to the writer's candidate set. This policy does not demand a search through arbitrary recompressions to discover every mathematically possible encoding of the same values.

I9 orders formats within one encoding. It does not minimize the required version of the reader code. A parent that uses an old ID can still have a child using a new ID. The complete tree determines compatibility. If a selected form is forbidden, the edition check fails. It does not instruct the serializer to try a newer form. Normal cumulative edition families preserve their earlier members, but arbitrary custom permission sets need not have that property.

5.3 Example: decimal parts

The historical decimal-byte-parts wire form stores one signed integer child. Its metadata field `lower_part_count` must be zero. A current array can also hold lower-part children for wider values. The plugin registers both historical and successor IDs and uses one current array implementation.

Consider decimal values whose scaled integers are 12, 34, and 56. If compression constructs a single signed child containing those integers, the plugin writes v1 and reuses the child. The current array implementation's ability to represent wider values does not force this array to use v2. The existing child can be serialized directly, with no upgrade, downgrade, or recompression.

If the constructed array contains lower-part children, the current plugin selects v2. The v1 reader's contract forbids those children even though it recognizes the metadata field's name. A hypothetical operation that merges children into one integer buffer needs its own value and cost analysis. The presence of small numerical values alone does not mean serialization already performs that operation. For an old target, a future scheme must directly construct the old single-child form when appropriate, or choose another permitted encoding.

5.4 Retaining old write paths (I8)

Let $B_{W,E}$ be the source-value domain for which writer W promises to write target E . I8 requires a retained construction procedure that, on that domain, terminates with a value-preserving representation whose complete serialization is permitted. This is an explicit availability obligation beyond Theorem 1.

A writer can satisfy I1 through I7 while deleting every old compression path and rejecting all writes to an old target. Reader compatibility remains true for successful writes but is then unhelpful to that writer's users. I8 rules out that regression on its stated domain. It makes no promise for arbitrary custom arrays or arbitrary source values outside that domain.

6 Future scheme configuration and recompression

6.1 Configuration obligations

Fix a target selection E . A scheme configuration c includes the behavior that affects its output representation. Each scheme orders its supported behaviors and selects the newest one whose declared output capabilities are permitted. Distinct algorithms can still compete. Their ordinary compression decisions are not ordered by wire-version age.

Let $P(c)$ be a set of typed IDs bounding the complete serialization of any successfully produced representation under configuration c , including every child and fallback. The future contract requires:

1. **Capability soundness:** for every successful compression result a , serialization terminates successfully without further compression: $S_W(a) = t \neq \perp$, with $U(t) \subseteq P(c)$.
2. **Target admissibility:** $P(c) \subseteq A(E)$.
3. **Phase consistency:** estimation, sample compression, and full compression use the same behavior configuration c . An estimate concerns that behavior. It need not predict the exact full-input compression ratio.
4. **Recursive closure:** child compressors and fallbacks use admissible configurations, and their possible IDs are included in $P(c)$. Configuration remains fixed for the write, or an equivalent coherent snapshot is used.
5. **Semantic preservation:** full compression preserves the input values. This does not follow from capability declarations.

Capability soundness is a universal obligation across the full input domain. A sample containing only narrow decimal values cannot justify using a v1-only claim for a full-input path that can produce lower-part children. The configured full path must handle that case with a permitted construction or report failure. An estimate alone cannot prove this property.

6.2 Theorem 5: configured compression needs no recompression to satisfy editions

Suppose the obligations above hold and full compression of source x under c succeeds with representation a . Its serialization succeeds without recompression solely to satisfy E , passes all edition checks, and is readable with the meaning of x by every conforming supporting reader.

Proof. Capability soundness provides the terminating serialization $t = S_W(a)$ without further compression. It also gives $U(t) \subseteq P(c)$. Admissibility gives $P(c) \subseteq A(E)$, hence $U(t) \subseteq A(E)$. All edition checks therefore pass. Semantic preservation equates the meaning of a with that of x . Apply Theorem 1. $\square$

The theorem does not infer absence of recompression merely from an allowlist: it uses the explicit construction guarantee in capability soundness. Its practical purpose is to require schemes to establish that guarantee before compression rather than repair incompatible arrays later. Stateful or configurable schemes are possible implementations, not a requirement to put a version field on every array.

Phase consistency additionally guarantees that candidates are evaluated under the behavior actually selected for compression. Without it, full compression can still produce permitted output, but the selection process can estimate one representation and produce another. No theorem here establishes optimal compression or estimator accuracy.

Array schemes cover the array dependencies they construct. A complete file also needs admissible layout, dtype, and aggregate construction. Their permission checks remain independent boundaries. A caller-supplied array need not have been built under c . It can require explicit adaptation, recompression, or rejection. This theorem does not extend to it without the same premises.

7 Implementation locations and remaining work

These source locations show where the model's requirements apply to the Rust implementation. They identify the APIs involved, but do not prove that every component satisfies the requirements.

- `vortex-array/src/array/plugin.rs` separates the in-memory plugin ID from its historical `serialized_ids`. Serialization returns a concrete ID, metadata, buffers, and children. Deserialization receives the exact wire ID.
- `vortex-edition/src/lib.rs` and `session.rs` define typed component membership, independently versioned families, cumulative inclusion, origin metadata, and selection.
- `vortex-btrblocks/src/builder.rs` filters schemes by their declared serialized IDs. General edition-derived behavior configuration, including phase consistency and recursive capability coverage, remains an implementation obligation for Theorem 5.
- `encodings/decimal-byte-parts/src/decimal_byte_parts/plugin/` implements the structural example. With no lower-part children, the plugin selects v1. Otherwise it selects v2. Both deserialize through the current representation.

Historical contract tests need to cover validation against the exact wire ID, every component kind, and recursively created children. Writers must retain the code needed for supported older targets. Scheme configuration needs sound output declarations, with fixtures covering files produced by new writers for old readers. These checks supply implementation evidence. The model separately requires preservation of values, compatibility of successful writes, and the ability to write every input in the promised domain.

The intended API direction is recorded in Vortex PR 9779. Updating the Vortex crates does not by itself require an upgrade or downgrade of an array, and edition enforcement does not choose the wire form.

8 Complete two-format matrix

This matrix applies the planned scheme configuration to one encoding in an otherwise compatible file. It covers eight writer, target, and output combinations, with two reader outcomes each. The wire-format column describes a possible output, not a separate scheme setting. A supported write means that suitable valid inputs and output shapes exist. It does not promise success for every input. L1 reading a v1-only E2 file does not establish that L1 supports all of E2.

Symbol	Meaning
v1	The encoding's original serialized format.
v2	A successor serialized format with a distinct wire ID.
L1	Older Vortex crate version that reads and writes v1 only.
L2	Newer Vortex crate version that reads and writes v1 and v2 through one current array implementation.
E1	Edition permitting v1.
E2	Edition permitting v1 and v2.
✓	Supported for a suitable input and correctly registered implementations.
X	Unsupported or forbidden.
N/A	No successful write, so no reader outcome.

Writer	Target	Wire form	Write ¹	L1 reads	L2 reads
L1	E1	v1	✓	✓	✓ ²
L1	E1	v2	X: unsupported and forbidden	N/A	N/A
L1	E2 ³	v1	✓	✓	✓ ²
L1	E2 ³	v2	X: unsupported	N/A	N/A
L2	E1	v1	✓ ⁴⁵	✓	✓ ²
L2	E1	v2	X: forbidden	N/A	N/A
L2	E2	v1	✓ ⁴⁵	✓	✓ ²
L2	E2	v2	✓ ⁴	X: unknown ID	✓

¹ Success is conditional on representability and successful preparation. Theorem 1 does not make all inputs writable. A permitted parent is insufficient if any child is forbidden.

² L2 decodes the historical contract directly into its current implementation. It need not first construct and upgrade a legacy array.

³ L1 must know the E2 declaration to select it. Registration grants permission, not a v2 implementation. A v1-only file can be read by L1 even though E2 also permits v2.

⁴ Future configurable schemes establish Theorem 5's obligations. Current BtrBlocks filtering uses schemes' declared serialized output IDs. That filtering is correct for its current declarations. A scheme that can produce several formats still needs configuration to select compatible behavior.

⁵ A newer implementation can construct the original single-child decimal form and emit v1 directly. For E1, a compatible construction must be selected before compression. For E2, the same old form can arise naturally, and I9 still selects v1.

9 Counterexamples when an invariant is omitted

Each example shows a failure that the omitted invariant prevents. These are hypothetical failures, not claims about current Vortex bugs.

Omitted invariant	What can change
I1: immutable contract	An ID starts interpreting a buffer as unsigned instead of signed. Old and new readers accept the same bytes but disagree on values.
I2: correct local reader	A registered decoder reverses the child order. All ID checks pass, but composition changes the values.
I3: correct serializer	A serializer truncates a wide value while claiming a valid old ID. Every reader consistently returns the wrong value.
I4: recursive enforcement	A permitted dictionary parent contains a forbidden compressed child. The target reader can dispatch the parent but cannot read the child.
I5: fixed edition membership	An already published edition gains v2. Its name no longer denotes the permission set users previously selected, even if its minimum version is also revised to preserve coverage. Alternatively, changing only a minimum version changes the published deployment promise without changing any file bytes.
I6: retained readers	The origin supplied v1 at its recorded minimum version but a later version removes it. Updating the reader's code breaks historical files.
I7: sound minimum version	The declared minimum predates the first v2 implementation. Membership is fixed and readers are additive, yet the promised minimum cannot read all output.
I8: retained writer paths	The writer keeps historical decoders but deletes old construction behavior. Old files remain readable, while supported old-target writes become unavailable.
I9: oldest-form choice	An array fitting v1 is always emitted as v2. The write can remain safe for E2, but an otherwise unnecessary newer-reader requirement is introduced.
I10: coherent schemes	Sampling uses the old form, while full compression creates a forbidden successor or child. Final validation remains safe by rejecting the file, but successful writing now requires another compression pass or a different construction.